CS342 · VISUAL PROGRAMMING

Chapter 1 OOP & Exceptions

ملخص الميد

محارب

محتوى الملف

01 الكلاس والأوبجكت

02 الوراثة Inheritance

03 Polymorphism

04 Casting

05 Packages و final و abstract

06 Interfaces

07 Exceptions: الفكرة والأنواع

08 throws و throw و try-catch

09 finally و Rethrow

10 Custom exceptions

11 طريقة حل Find Output

01 الكلاس والأوبجكت

A **class** describes the structure and behavior of a set of similar objects. A specific instance of a class is called an **object**.
An object has a unique identity, **state** (data fields with their current values) and **behaviors** (methods).

الكلاس هو القالب، والأوبجكت نسخة حقيقية منه. الـ **state** هي قيم المتغيرات، والـ **behavior** هي الدوال.

الخانة في Class diagram	وش فيها
فوق	اسم الكلاس
النص	الـ attributes (المتغيرات)
تحت	الـ operations (الدوال)

```
Student s;                  // declaration
s = new Student();          // creation: calls the constructor
s.name = "Ali";             // access a public field with the dot
s.setName("Ahmad");         // call a method
```

مثال السلايد: كلاس فيه أكثر من constructor (نفس الاسم وباراميترات مختلفة)، و **this** تعني الأوبجكت الحالي.

```
public class Holiday {
    private String name;
    private int day;
    private String month;
    public Holiday(String name, int day, String month) {
        this.name = name; this.day = day; this.month = month;
    }
    public Holiday(String name, String month) {
        this.name = name; this.month = month; this.day = 7;
    }
    public Holiday(String name) {
        this.name = name; this.day = 23; this.month = "September";
    }
    public boolean sameMonth(Holiday h) {
        return this.month.equals(h.month);
    }
    double avgDay(Holiday[] hol) {
        int sum = 0;
        for (int i = 0; i < hol.length; i++) sum += hol[i].day;
        return ((double) sum / hol.length);
    }
}
```

```
hd[0] = new Holiday("Vacation1", 6, "November");   // day 6
hd[1] = new Holiday("Vacation2", "November");      // day 7
hd[2] = new Holiday("Vacation3");                  // day 23, September
hd[0].sameMonth(hd[2])   →  November vs September  →  not equal
hd[2].avgDay(hd)         →  (6 + 7 + 23) ÷ 3 = 12.0
```

02 الوراثة Inheritance

```
public class Lawyer extends Employee {
    public Lawyer(int years) {
        super(years);                // calls the Employee constructor
    }
    @Override
    public double getSalary() {      // override: a new version in the subclass
        double baseSalary = super.getSalary();   // calls the parent method
        return baseSalary + 5000.00;
    }
}
```

الكلمة	وش تسوي
extends	الكلاس يرث من كلاس ثاني (superclass)
override	تكتب نسخة جديدة من دالة الأب داخل الابن
@Override	تقول لـ Java إن الدالة موجودة في الأب، وJava تتأكد إن الـ override صحيح
super.method()	تنادي دالة الأب
super(params)	تنادي constructor الأب

The class **Object** forms the root of the overall inheritance tree of all Java classes. Every class is implicitly a subclass of Object. It defines methods such as **toString()**, which returns a text representation of the object, usually so that it can be printed.

يعني **System.out.println(obj)** يطبع اللي ترجعه **toString()**. ولو كتبتها في كلاسك، تطلع نسختك أنت.

instanceof يسأل: هل المتغير يشير لأوبجكت من هالنوع؟ ويرجع true أو false.

```
String s = "hello";   Point p = new Point();
s instanceof Point    →  false        p instanceof String  →  false
s instanceof String   →  true         p instanceof Object  →  true
p instanceof Point    →  true         s instanceof Object  →  true
null instanceof String →  false       null instanceof Object →  false
```

Polymorphism 03

Polymorphism: ability for the same code to be used with different types of objects and behave differently with each.

نفس السطر، بس كل أوبجكت ينفذ نسخته هو. الأساس هو الـ override.

```
Animal someanimal;
someanimal = somecat;   someanimal.display();   // display() of Cat
someanimal = somebird;  someanimal.display();   // display() of Bird
for (int j = 0; j < 200; j++)
    someanimal[j].display();   // no need to know if it is a Cat or a Bird
```

القاعدة: Java تختار الدالة حسب **نوع الأوبجكت الحقيقي** (اللي بعد **new**)، مو حسب نوع المتغير.

Casting 04

	Upcasting	Downcasting
الاتجاه	Dog → Animal	Animal → Dog
الكتابة	بدون cast (implicit)	لازم cast صريح **(Dog)**
الخطر	آمن دايم	ممكن run-time error

Objects once created always know their type. The object itself is NOT changed or converted: casting is telling the compiler to ignore the "type mismatch". You need to cast downwards to use methods defined only in the derived class.

```
Animal a = new Cat(...);
a.motion();              // syntax error: motion() is not a method of Animal
((Cat) a).motion();      // OK
Cat c = a;               // syntax error: can't assign a superclass to a subclass
Cat c = (Cat) a;         // OK
Animal b = new Bird(...);
Cat c2 = (Cat) b;        // compiles, but RUN-TIME ERROR (ClassCastException)
if (b instanceof Cat)    // check first to stay safe
    ((Cat) b).motion();
```

هذا بالضبط سؤال Find Output في ميد 471. اسأل نفسك عن كل سطر سؤالين:

1. **وقت الترجمة:** هل الدالة موجودة في نوع المتغير (أو النوع اللي سويت له cast)؟ إذا لا، يطلع syntax error.
2. **وقت التشغيل:** هل الأوبجكت الحقيقي من نوع الـ cast أو ابن له؟ إذا لا، يطلع **ClassCastException** والبرنامج يوقف عند هالسطر.

05 Packages و final و abstract

A **package** name is defined by using the keyword package as the first instruction. To use a class from another package: **import**.

```
package it.infn.ts;        // first line of the file
import it.infn.ts.Book;    // use Book from another package
```

Modifier	على الكلاس	على الدالة
final	ما أحد يقدر يورث منه	ما ينفع لها override
abstract	ما تقدر تسوي منه أوبجكت (new)	بدون body، ولازم الابن يكتبها

abstract تقريباً عكس **final**: تسوي تصميم عام والأبناء يكملونه.

Interfaces 06

An **interface** describes *what* classes should do, without specifying *how* they should do it.

- all fields are static and final
- all methods have no body and are public
- no instances can be created from interfaces
- a class can implement more than one interface

```
interface NiceBehavior {
    public String getName();
    public void sayGoodBye();
}
class IOSerialBoard2 implements IOBoardInterface, NiceBehavior {
    ...   // must implement ALL the methods of BOTH interfaces
}
```

سؤال السلايد "Which methods should it implement?" جوابه: كل دوال الـ interfaces الاثنين.

	abstract class	interface
الكلمة	**extends**	**implements**
كم واحد	واحد بس	أكثر من واحد
الدوال	ممكن فيها body وممكن abstract	كلها بدون body وpublic
new	ممنوع	ممنوع

07 Exceptions: الفكرة والأنواع

When a program runs into a runtime error, the program terminates abnormally. Exception handling lets the program continue to run or terminate gracefully. It enables a method to throw an exception to its caller.

```
Object
 └── Throwable
      ├── Error  (LinkageError, VirtualMachineError ...)        → unchecked
      └── Exception (IOException, ClassNotFoundException ...)    → checked
           └── RuntimeException                                  → unchecked
                (ArithmeticException, NullPointerException,
                 IndexOutOfBoundsException, IllegalArgumentException ...)
```

النوع	وش هو
Error	أخطاء النظام من الـ JVM، نادرة وما تقدر تسوي لها شي غير إنك تنبه المستخدم وتقفل
Exception	أخطاء من برنامجك أو من ظروف خارجية، وتقدر تمسكها وتعالجها
RuntimeException	أخطاء برمجة: cast غلط، index برا المصفوفة، أخطاء حسابية

Unchecked exceptions: RuntimeException, Error and their subclasses. Java does not force you to catch them.
Checked exceptions: all the others. The compiler forces the programmer to check and deal with them.

Exception	متى تطلع
NumberFormatException	**Integer.parseInt("x")**: النص مو رقم، أو فاضي، أو فيه فاصلة مثل "2.3"
ArithmeticException	قسمة int على صفر
NullPointerException	تستخدم متغير ما فيه أوبجكت (null)
IndexOutOfBoundsException	index برا حدود المصفوفة
ClassCastException	downcast لنوع غلط
InputMismatchException	Scanner يقرأ نوع غير المتوقع

08 throws و throw و try-catch

```
method1() {                          method2() throws Exception {   // declare
    try {                                if (an error occurs) {
        invoke method2;                      throw new Exception();     // throw
    }                                    }
    catch (Exception ex) {           }
        Process exception;           // catch
    }
}
```

الخطوة	الكتابة	المعنى
Declare	**throws**	في رأس الدالة: هالدالة ممكن ترمي هالنوع
Throw	**throw new X()**	ترمي الخطأ لما تكتشفه
Catch	**try {} catch (X e) {}**	تمسكه وتعالجه

انتبه: **throws** (بـ s) في رأس الدالة، و **throw** (بدون s) داخل الكود.

```
public void setRadius(double newRadius) throws IllegalArgumentException {
    if (newRadius >= 0)
        radius = newRadius;
    else
        throw new IllegalArgumentException("Radius cannot be negative");
}
```

Catch or declare: if p2 may throw a checked exception (e.g. IOException), p1 must either call it inside try-catch (a) or declare throws IOException (b).

```
(a) void p1() {                          (b) void p1() throws IOException {
        try { p2(); }                            p2();
        catch (IOException ex) { ... }       }
    }
```

ترتيب الـ catch: Java تجرب الـ catch من فوق لتحت، وتدخل **أول واحد يناسب** وتطنش الباقي. وبعد ما يخلص الـ catch يكمل البرنامج بعد الـ try كله، مو من السطر اللي صار فيه الخطأ.

09 finally و Rethrow

```
try {
    statements;
}
catch (TheException ex) {
    handling ex;
    throw ex;          // rethrow: control goes to the caller
}
finally {
    finalStatements;   // ALWAYS executed
}
Next statement;
```

الحالة	وش يتنفذ
ما صار خطأ	try كامل → finally → Next statement
خطأ وفيه catch له	try لين الخطأ → catch → finally → Next statement
catch يسوي **throw ex**	try → catch → finally → يرجع للي نادى الدالة، و Next statement ما يتنفذ
خطأ ما له catch	try → finally → يرجع للي نادى الدالة

مثال متتبّع مع ناتجه:

```
static void m(int x) throws Exception {
    try {
        System.out.println("A");
        if (x == 1) throw new ArithmeticException();
        if (x == 2) throw new Exception("boom");
        System.out.println("B");
    }
    catch (ArithmeticException e) { System.out.println("C"); }
    finally { System.out.println("D"); }
    System.out.println("E");
}
// main calls m(0), m(1), m(2), each inside try/catch printing "main: " +
e.getMessage()
```

```
m(0):  A  B  D  E
m(1):  A  C  D  E
m(2):  A  D  main: boom
```

Custom exceptions 10

Use the exception classes in the API whenever possible. Define custom exception classes if the predefined classes are not sufficient, by extending Exception or a subclass of Exception.

```
public class ExceptionRange extends Exception {
    public ExceptionRange(String s) {
        super(s);                          // the message, read later with getMessage()
    }
}
public class ExceptionEmpty extends Exception {
    public String toString() {
        return ("No note is valid");     // printed by System.out.println(er)
    }
}
```

تكتب	يطبع
e.getMessage()	النص اللي انرسل لـ **super(s)**
System.out.println(e)	اللي ترجعه **toString()**

When to use exceptions: for unexpected error conditions, not for simple expected situations. Exception handling requires more time and resources (a new exception object, rolling back the call stack, propagating the error).

```
try { System.out.println(refVar.toString()); }          // avoid
catch (NullPointerException ex) { System.out.println("refVar is null"); }
if (refVar != null) System.out.println(refVar.toString());   // better
else System.out.println("refVar is null");
```

11 طريقة حل Find Output

1. ابدأ من **main**، وامشِ سطر سطر، واكتب كل **println** أول ما يتنفذ.

2. داخل الـ loop: كل لفة لحالها. الخطأ يوقف **اللفة**، والـ catch اللي داخل الـ loop يخلي الـ loop يكمل.

3. **Integer.parseInt**: أي شي مو عدد صحيح يطلع **NumberFormatException**: حروف، أو " "، أو "2.3".

4. الـ cast: شيك على النوع الحقيقي للأوبجكت (اللي بعد **new**).

5. لو صار خطأ ما أحد مسكه، البرنامج يوقف عند هالسطر، وكل اللي بعده ما يطبع.